

Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей
Кафедра программного обеспечения информационных технологий
Дисциплина: Компьютерные системы и сети

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
на тему:

**Приложение, реализующее сетевую систему мониторинга
ресурсов и состояния узлов**

БГУИР КП 6-05-0612-01 016 ПЗ

Студент

ст. гр. 410902
Большакова М.В.

Руководитель

Болтак С.В.

Минск 202

СОДЕРЖАНИЕ

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	5
1.2 Обзор систем мониторинга	5
1.3 Постановка задачи	6
2 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА	8
2.1 Структура программы.....	8
2.2 Выбор линии связи и структуры сигналов	8
2.3 Проектирование интерфейса программного средства	9
2.4 Проектирование функционала программного средства.....	11
2.5 Подключение агента к серверу	11
2.6 Сбор и отправка метрик	11
2.7 Обработка данных на сервере.....	12
3 АЛГОРИТМ ФУНКЦИОНИРОВАНИЯ СИСТЕМЫ.....	13
3.1 Реализация агента	13
3.2 Реализация сервера	13
3.3 Организация сетевого взаимодействия.....	13
3.4 Приём и обработка сообщений.....	14
3.5 Обновление графического интерфейса.....	14
3.6 Организация хранения данных в SQLite.....	14
4 ТЕСТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА.....	16
4.1 Анализ процесса установки соединения в Wireshark	21
4.2 Анализ отказоустойчивости системы (переподключение)	21
4.3 Анализ завершения сеанса связи	22
5 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ.....	23
5.1 Системные и сетевые требования.....	23
5.2 Настройка сетевого взаимодействия и параметров сокетов	23
5.3 Порядок запуска	23
5.4 Интерфейс программного средства.....	24
5.4.1 Главное окно.....	24
5.4.2 Вкладка «Таблица узлов»	25
5.4.3 Вкладка «Графики»	26
5.4.4 Вкладка «Процессы».....	27
ЗАКЛЮЧЕНИЕ	29
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	30
ПРИЛОЖЕНИЕ А.....	31
ПРИЛОЖЕНИЕ Б.....	39

ВВЕДЕНИЕ

Информационные технологии и компьютерные сети являются неотъемлемой частью функционирования современных организаций и учебных заведений. С ростом количества сетевых узлов и усложнением инфраструктуры критически важной задачей становится обеспечение их стабильной и бесперебойной работы.

Одним из ключевых инструментов управления ИТ-инфраструктурой являются системы сетевого мониторинга. Мониторинг позволяет в режиме реального времени отслеживать состояние аппаратных ресурсов, таких как загрузка центрального процессора (*CPU*), использование оперативной памяти (*RAM*) и состояние дискового пространства (*Disk*). Своевременное получение данных о критических нагрузках позволяет предотвратить отказы оборудования и оптимизировать распределение вычислительных мощностей.

История развития средств мониторинга прошла путь от простых консольных утилит до сложных распределенных комплексов с графическим интерфейсом. Современные решения базируются на архитектуре «клиент-сервер» (или «агент-сервер»), где сбор данных осуществляется непосредственно на целевом узле, а визуализация и анализ – на центральном сервере администратора.

Для обеспечения надежного взаимодействия между компонентами системы используются стандартизированные протоколы передачи данных (*TCP/IP*) и универсальные форматы обмена информацией, такие как *JSON*. Важным этапом разработки таких систем является верификация сетевого трафика с помощью специализированного ПО, например, анализатора *Wireshark*, что позволяет гарантировать точность передаваемых метрик.

Тема разработки систем мониторинга является актуальной ввиду потребности в легковесных и кроссплатформенных решениях, способных работать в гетерогенных сетях. Целью данного курсового проекта является разработка программного средства, реализующего сетевую систему мониторинга ресурсов и состояния узлов.

Для достижения поставленной в курсовой работе цели необходимо решить следующий ряд последовательных задач:

- спроектировать клиент-серверную архитектуру программного средства на основе парадигмы «агент-сервер»;
- реализовать легковесную агентскую часть для автономного сбора и передачи системных метрик;
- разработать многопоточную серверную часть с современным графическим интерфейсом;
- обеспечить сохранение данных метрик;
- провести всестороннее комплексное тестирование разработанного программного продукта, включая имитацию обрывов связи;
- составить подробное руководство пользователя и администратора системы.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.2 Обзор систем мониторинга

На сегодняшний день существует огромное количество программных средств для сетевого мониторинга ресурсов. Каждое из них обладает своим уникальным функционалом и предназначено для решения задач определенного масштаба. В первую очередь стоит рассмотреть популярное решение корпоративного уровня – систему *Zabbix*. Внешний вид данного приложения представлен на рисунке 1.1.

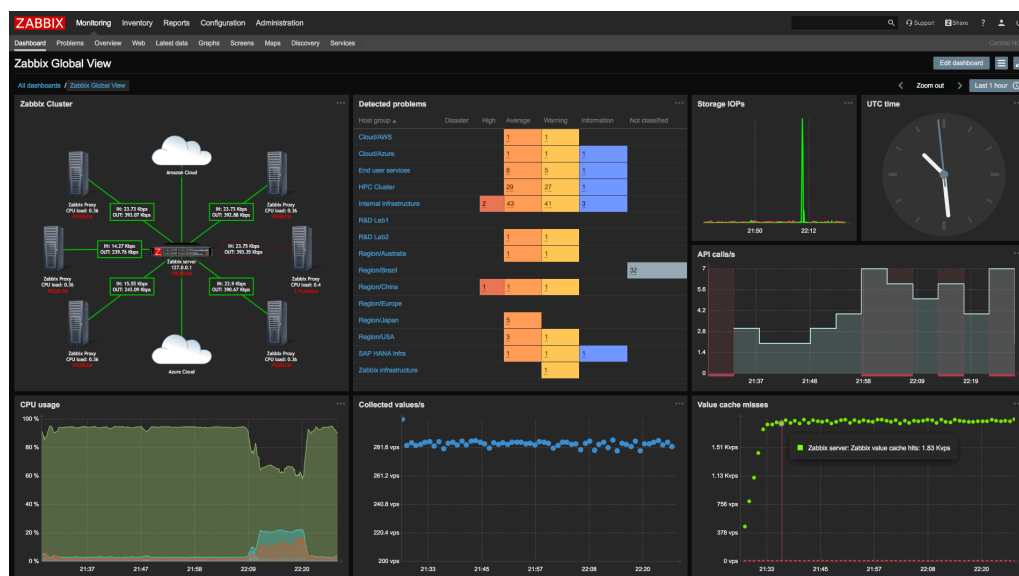


Рисунок 1.1 – Интерфейс *Zabbix*

Данное программное обеспечение имеет открытый исходный код и позволяет детально отслеживать динамику работы серверов и сетевого оборудования. Сбор данных в *Zabbix* чаще всего осуществляется посредством установки специализированных агентов на целевые узлы. Самая важная особенность приложений и систем такого типа – это гибкая настройка триггеров и оповещений.

Однако для небольших локальных сетей этот инструмент обладает избыточной архитектурой: он требует развертывания отдельного полноценного веб-сервера (на базе *PHP*) и реляционной базы данных (*MySQL* или *PostgreSQL*).

Далее стоит рассмотреть популярную на сегодняшний день систему мониторинга *Prometheus*. Внешний вид данного приложения представлен на рисунке 1.2.

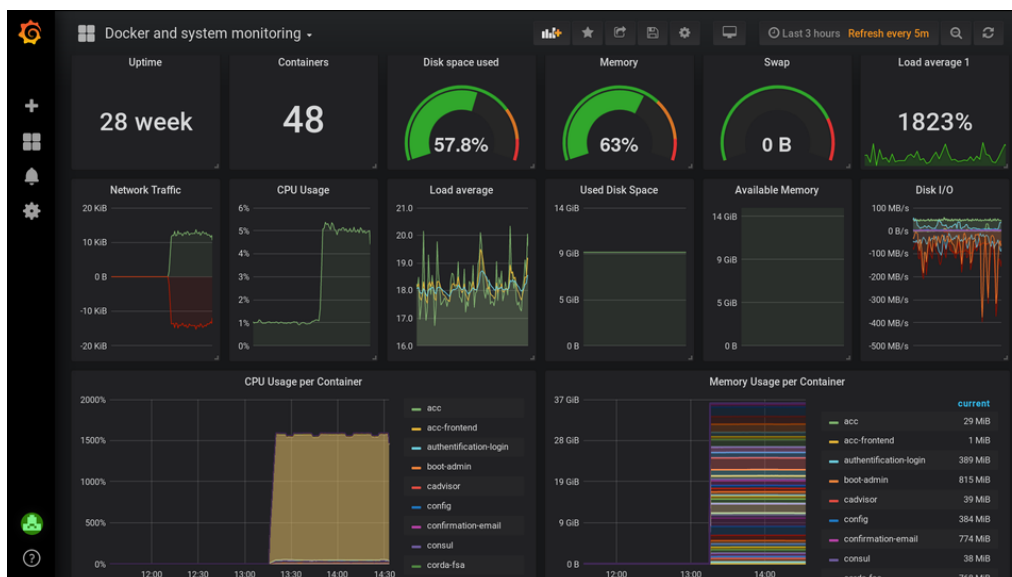


Рисунок 1.2 – Интерфейс *Prometheus*

Данная система за годы своего развития сумела определить новые стандарты в сегменте мониторинга облачных инфраструктур. В отличие от классических систем, *Prometheus* по умолчанию использует *pull*-модель (сервер самостоятельно опрашивает узлы по протоколу *HTTP*). Главным недостатком данного решения для небольшой вычислительной сети является то, что *Prometheus* не имеет встроенных мощных средств визуализации и требует использования сторонних инструментов для построения графического интерфейса (таких как *Grafana*). Развертывание такой связки требует дополнительных вычислительных ресурсов оперативной памяти.

Исходя из проведенного анализа, можно сделать вывод, что использование масштабируемых корпоративных аналогов для отслеживания базовых метрик (*CPU*, *RAM*, *Disk*) нерационально. Требуется разработка собственного легковесного программного средства, которое будет потреблять минимум ресурсов целевой системы.

1.3 Постановка задачи

В рамках данной курсовой работы планируется разработать программное средство, реализующее сетевую систему мониторинга ресурсов и состояния узлов. Система должна всегда функционировать в режиме реального времени, обеспечивая гарантированный непрерывный сбор, агрегацию, сериализацию и визуальное отображение метрик производительности одновременно с нескольких сетевых узлов.

В процессе разработки должны быть реализованы следующие базовые функциональные возможности программного агента (клиентской части):

- автономный сбор системных метрик целевых серверных узлов, включая общую нагрузку на все ядра *CPU*, процент использования *RAM* и состояние системного раздела жесткого диска;

- сбор подробного списка ресурсоемких процессов, работающих в операционной системе, с сортировкой по потреблению *CPU* и *RAM*;
- быстрая и эффективная сериализация всех собранных метрик в стандартизированный текстовый формат обмена данными *JSON*;
- надежная отправка сформированных пакетов с метриками на выделенный сервер по локальной или глобальной сети исключительно через потоковые *TCP*-сокеты;
- обработка любых возникающих ошибок сетевого уровня с повторными попытками переподключения.

Также в обязательном порядке должны быть реализованы основные программные возможности серверной части инфраструктуры:

- асинхронная или многопоточная работа приложения в режиме прослушивания распределенной сети без блокировки графического интерфейса;
- прием, буферизация и безопасная десериализация входящих байтовых потоков данных от множества подключенных агентов;
- мгновенное отображение полученных и обработанных данных в удобном графическом пользовательском интерфейсе, реализованном на базе библиотеки *PyQt5*;
- отрисовка динамических графиков изменения аппаратной нагрузки в реальном времени с применением *PyQtGraph*;
- строгое логирование и сохранение всей истории поступающих метрик в локальные файлы;
- внедрение системы визуального оповещения (динамическая смена цветовых индикаторов статуса) при превышении заранее заданных критических пороговых значений нагрузки.

Дополнительно в рамках проектирования требуется обеспечить кроссплатформенность приложения (гарантированная работа как на базе ОС *Windows*, так и на *macOS*), а также использовать анализатор сетевых протоколов *Wireshark* для отладки и верификации передаваемых сетевых пакетов.

Для разработки программного средства будет использоваться язык программирования *Python* версии 3.14.3 и среда разработки *VS Code*. Графический интерфейс будет реализован с применением фреймворка *Qt* для *Python*, а сетевое взаимодействие – посредством сетевых модулей (*socket*, *asyncio* или *requests*).

2 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА

2.1 Структура программы

При разработке приложения будет использована клиент-серверная архитектура в режиме «агент-сервер». Система разделена на две логические части:

- агент – устанавливается на целевые серверные узлы, осуществляет сбор системных метрик и отправляет их по сети;
- сервер – принимает входящие подключения от агентов, агрегирует информацию и выводит ее на экран пользователя.

При разработке программного комплекса будет использовано 10 модулей, разделённых на клиентскую (агент) и серверную части.

Модули серверной части:

- 1 *MonitorApp* – модуль, предназначенный для отображения главного окна приложения и управления вкладками;
- 2 *ServerThread* – модуль, обеспечивающий прослушивание сети и приём входящих подключений от агентов;
- 3 *ClientHandlerThread* – модуль, обеспечивающий обработку данных от одного агента в отдельном потоке;
- 4 *NodeChartWidget* – модуль, обеспечивающий построение живых графиков загрузки ресурсов в реальном времени;
- 5 *DatabaseManager* – модуль, отвечающий за хранение истории метрик в базе данных *SQLite*;
- 6 *HistoryTab* – модуль, предназначенный для просмотра истории метрик из базы данных.

Модули клиентской части (агент):

- 1 *start_agent* – основная функция агента, координирующая сбор и отправку данных;
- 2 *collect_metrics* – функция сбора системных метрик с использованием библиотеки *psutil*;
- 3 *serialize_json* – функция преобразования объектов с метриками в формат *JSON*;
- 4 *send_data* – функция установки соединения с сервером и передачи данных через сокеты.

2.2 Выбор линии связи и структуры сигналов

В качестве линии связи используется локальная вычислительная сеть на базе стека протоколов *TCP/IP*. Взаимодействие между узлами реализуется с помощью встроенных сетевых библиотек Python (*socket*). В качестве формата передачи данных выбран формат *JSON*. Данный текстовый формат обеспечивает лёгкость сериализации и десериализации объектов, а также читаемость при отладке.

Каждый пакет данных завершается символом-разделителем «\n», что позволяет серверу корректно определять границы отдельных сообщений при потоковой передаче по *TCP*.

2.3 Проектирование интерфейса программного средства

Графический пользовательский интерфейс (*GUI*) серверной части спроектирован и разработан с применением библиотеки *PyQt5*. Интерфейс концептуально организован в виде эргономичной системы вкладок (виджет *QTabWidget*), что гарантирует интуитивно понятную и быструю навигацию между различными режимами просмотра поступающих данных.

Первая базовая вкладка — «Таблица узлов» — содержит динамическую сводную таблицу (*QTableWidget*) всех успешно подключенных на данный момент агентов. Каждая создаваемая строка в таблице жестко привязывается к одному физическому узлу и отображает: цветовой *LED*-подобный индикатор системного статуса, имя узла-отправителя, его внешний *IP*-адрес, цветные прогресс-бары текущей загрузки *CPU*, *RAM* и системного диска, а также точную временную метку последнего обновления пакета. Макет данной вкладки представлен на рисунке 2.1.

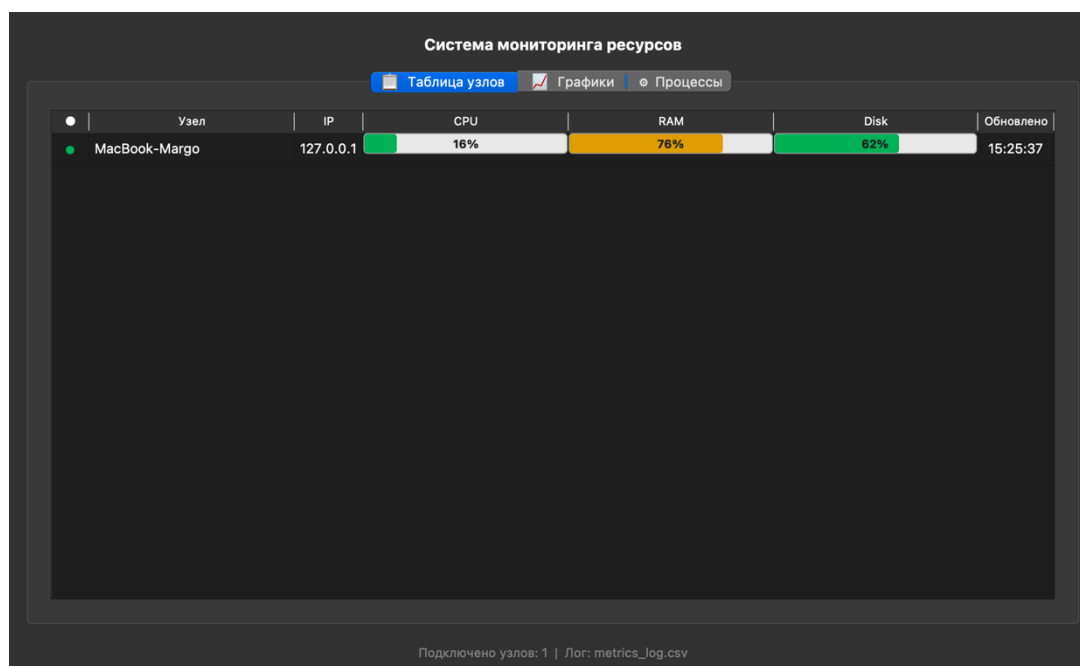


Рисунок 2.1 – Макет вкладки «Таблица узлов»

Вторая вкладка — «Графики» — предназначена для построения и отрисовки живых графиков поступающих метрик для каждого подключенного клиента. Для каждого узла автоматически генерируются три независимых потоковых графика. Графики отрисовываются с помощью библиотеки *pyqtgraph*, которая использует аппаратное ускорение и способна рендерить сотни кадров в секунду без существенной нагрузки на процессор. Каждый

график оперирует кольцевым буфером на 60 последних значений. Макет вкладки «Графики» представлен на рисунке 2.2.

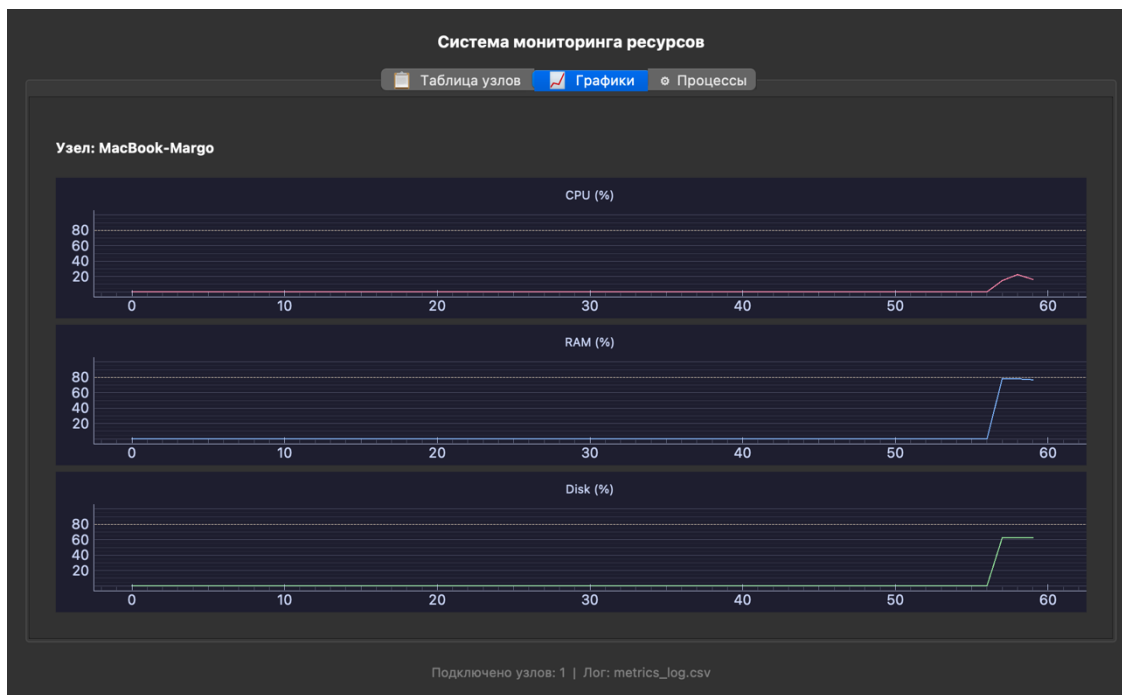


Рисунок 2.2 – Макет вкладки «Графики»

Третья опциональная вкладка — «Процессы» — предоставляет возможность детального просмотра дерева ресурсоемких процессов целевой операционной системы. Администратор может выбрать конкретный узел из выпадающего списка *QComboBox* и проанализировать, какое именно приложение потребляет всю доступную память или процессорное время. Макет вкладки «Графики» представлен на рисунке 2.3.

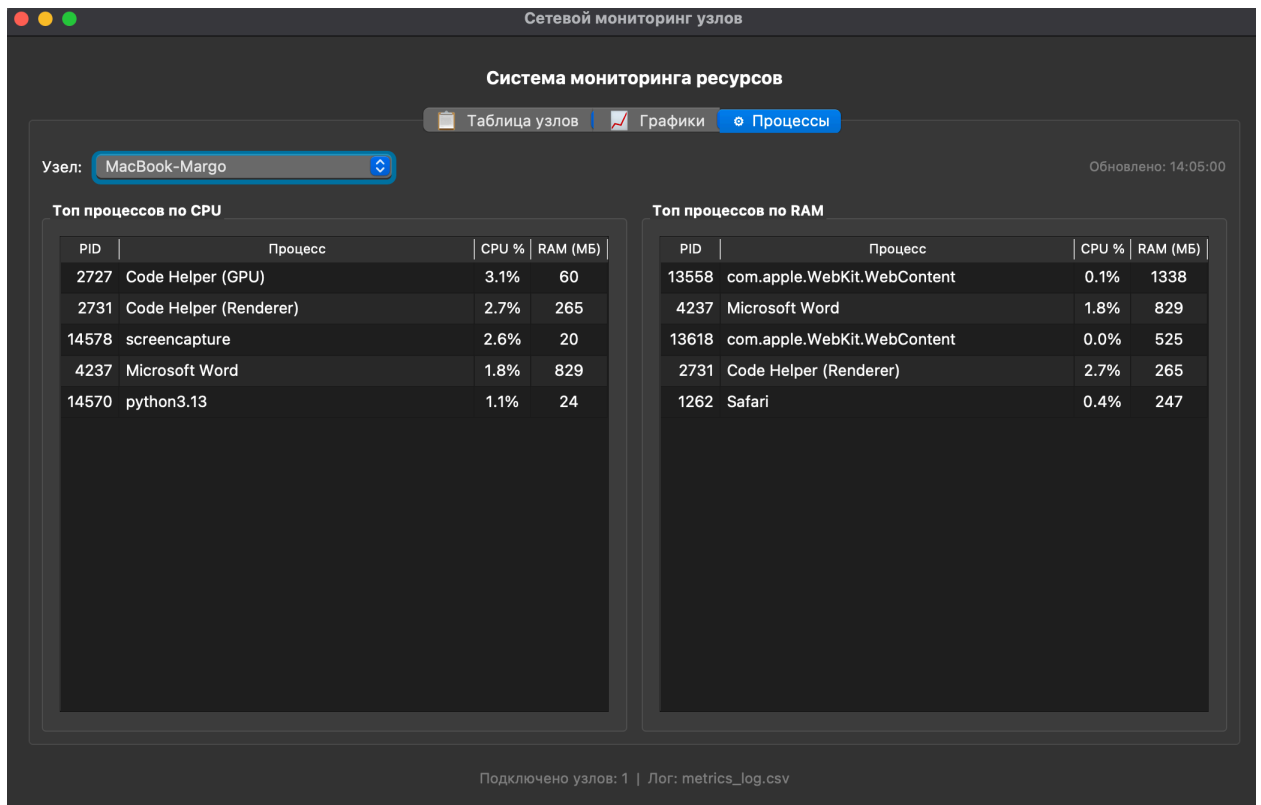


Рисунок 2.3 – Макет вкладки «Процессы»

2.4 Проектирование функционала программного средства

Грамотно поставленная задача и хорошо составленные алгоритмы – ключевая фаза в проектировании программного средства. Система мониторинга должна предоставлять пользователю следующий минимальный функционал:

- отображение актуальных метрик узлов в реальном времени;
- подключение и отображение нескольких агентов одновременно;
- сохранение и просмотр истории метрик.

2.5 Подключение агента к серверу

Подключение агента к серверу происходит автоматически при запуске программы агента. Агент создаёт *TCP*-сокеты и выполняет попытку соединения методом *connect()*. Если сервер недоступен, агент повторяет попытку подключения каждые 2 секунды. После успешного подключения агент переходит в режим циклической отправки метрик.

2.6 Сбор и отправка метрик

Сбор метрик происходит в бесконечном цикле с интервалом 2 секунды. На каждой итерации цикла вызываются методы библиотеки *psutil* для получения текущих значений загрузки *CPU*, использования *RAM* и занятости

дискового пространства. Полученные данные упаковываются в *JSON*-строку и отправляются на сервер через сокет.

2.7 Обработка данных на сервере

Обработка входящих данных на сервере происходит в отдельном потоке для каждого агента. При получении *JSON*-пакета сервер получает и десериализует данные, сохраняет их в базу данных *SQLite*, обновляет таблицу и графики в интерфейсе. Если значения метрик превышают пороговые значения, цветовой индикатор строки меняется с зелёного на красный.

3 АЛГОРИТМ ФУНКЦИОНИРОВАНИЯ СИСТЕМЫ

3.1 Реализация агента

Программный агент реализован в файле *agent.py* в виде консольного *Python*-приложения. Основой является функция *start_agent()*, которая создаёт *TCP*-сокеты и запускает бесконечный цикл сбора и отправки метрик.

Сбор метрик осуществляется с помощью библиотеки *psutil* – кроссплатформенного инструмента для получения системной информации. Метод *cpu_percent(interval=1)* возвращает среднюю загрузку процессора за одну секунду. Метод *virtual_memory().percent* возвращает долю используемой оперативной памяти. Метод *disk_usage('/').percent* возвращает процент занятого дискового пространства. Код, реализующий сбор метрик, представлен ниже.

```
data = {
    "node": "MacBook-Agent",
    "cpu": psutil.cpu_percent(interval=1),
    "ram": psutil.virtual_memory().percent,
    "disk": psutil.disk_usage('/').percent
}
```

Собранные данные формируются в словарь *Python*, включающий имя узла и значения метрик. Словарь сериализуется в *JSON*-строку с помощью функции *json.dumps()* и дополняется символом-разделителем «*\n*». Итоговая строка кодируется в байты *UTF-8* и передаётся на сервер методом *sendall()*. При потере соединения агент перехватывает исключение *BrokenPipeError* и завершает работу с соответствующим сообщением. Код отправки данных представлен ниже.

```
json_data = json.dumps(data) + "\n"
s.sendall(json_data.encode('utf-8'))
```

3.2 Реализация сервера

Серверная часть представляет собой многопоточное приложение, способное обрабатывать запросы от нескольких агентов одновременно без блокировки графического интерфейса. Архитектура сервера базируется на наследовании классов от *QThread* (фреймворк *PyQt5*).

3.3 Организация сетевого взаимодействия

Класс *ServerThread* создаёт *TCP*-сокеты с параметром *SO_REUSEADDR*, который позволяет перезапустить сервер без ожидания, привязывает его к адресу и запускает прослушивание. При поступлении нового подключения методом *accept()* создаётся объект *ClientHandlerThread*. Данный подход обеспечивает независимую обработку каждого агента и предотвращает

блокировку интерфейса. Код реализации многопоточного приёма подключений представлен ниже.

```
conn, addr = s.accept()
client = ClientHandlerThread(conn, addr)
self.new_client.emit(client)
client.start()
```

3.4 Приём и обработка сообщений

Класс *ClientHandlerThread* в методе *run()* реализует цикл чтения данных из сокета. Принятые байты декодируются и накапливаются в строковом буфере. При обнаружении символа-разделителя «\n» из буфера извлекается полный пакет, который парсится методом *json.loads()*. К метрикам добавляются IP-адрес агента и временная метка. Сформированный словарь передаётся в *GUI* через сигнал *data_received*. Код обработки входящего пакета представлен ниже.

```
buffer += data.decode('utf-8')
while '\n' in buffer:
    line, buffer = buffer.split('\n', 1)
    metrics = json.loads(line)
    self.data_received.emit(metrics)
```

3.5 Обновление графического интерфейса

Взаимодействие между сетевыми потоками и графическим интерфейсом реализовано через механизм сигналов и слотов *PyQt5*. *Qt* запрещает изменение элементов интерфейса из фонового потока напрямую. Сигнал передаёт данные в основной поток, где вызывается слот *update_metrics()* – метод, обновляющий таблицу и графики. Если узел встречается впервые, добавляется новая строка в таблицу и создаётся виджет *NodeChartWidget* с тремя живыми графиками.

3.6 Организация хранения данных в *SQLite*

Для сохранения истории метрик используется встроенная в *Python* база данных *SQLite*. Вся работа с базой данных инкапсулирована в классе *DatabaseManager*. При инициализации класс создаёт файл *metrics.db* и выполняет *SQL*-запрос *CREATE TABLE IF NOT EXISTS*, создающий таблицу *metrics*. Структура таблицы включает поля: *id* (автоинкремент), *timestamp* (дата и время), *node* (имя узла), *ip* (адрес агента), *cpu*, *ram*, *disk*.

Метод *insert()* вызывается при каждом поступлении нового пакета от агента. Для защиты от одновременной записи из нескольких потоков используется объект *threading.Lock()*. Для безопасной передачи параметров в *SQL*-запросы применяются *placeholder*-символы «?», что предотвращает *SQL*-инъекции. Код метода вставки записи представлен ниже.

```

conn.execute(
    "INSERT INTO metrics (timestamp, node, ip, cpu, ram,
disk) "
    " VALUES (?, ?, ?, ?, ?, ?)",
    (timestamp, node, ip, cpu, ram, disk)
)

```

Метод *get_last_records()* принимает имя узла и возвращает последние *N* записей с сортировкой от новых к старым (*ORDER BY id DESC*). Метод *get_total_count()* возвращает общее количество записей в таблице.

Таким образом, детальный анализ алгоритмов функционирования системы показывает, что разработанная архитектура полностью решает задачи надежного сбора, передачи и обработки телеметрии. Использование библиотеки *psutil* обеспечивает точный кроссплатформенный сбор метрик, а применение многопоточности на базе *QThread* позволяет серверу параллельно обслуживать нескольких агентов без блокировки графического интерфейса. Механизм сигналов и слотов *PyQt5* гарантирует безопасное обновление потоков и элементов *GUI* в реальном времени. В свою очередь, интеграция базы данных *SQLite* с механизмами блокировки потоков (*threading.Lock*) предоставляет надежное, упорядоченное и защищенное от параллельных коллизий хранилище для долгосрочного аудита состояния узлов сети.

4 ТЕСТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА

В ходе тестирования приложения проверялась корректность работы всех основных функций программного средства. Тестирование проводилось на компьютере *MacBook* под управлением *macOS*, что также подтвердило кроссплатформенность приложения.

Тест 1. Подключение одного агента.

Был запущен сервер, после чего инициирован запуск программного агента. Сервер успешно принял входящее *TCP*-соединение. На вкладке «Таблица узлов» динамически добавилась новая строка с идентификатором узла (например, *MacBook-Margo*), IP-адресом источника и актуальными показателями системных ресурсов. Индикаторы нагрузки (*Progress Bar*) корректно отображали значения в допустимых пределах (зеленый цвет). Представлено на рисунке 4.1.

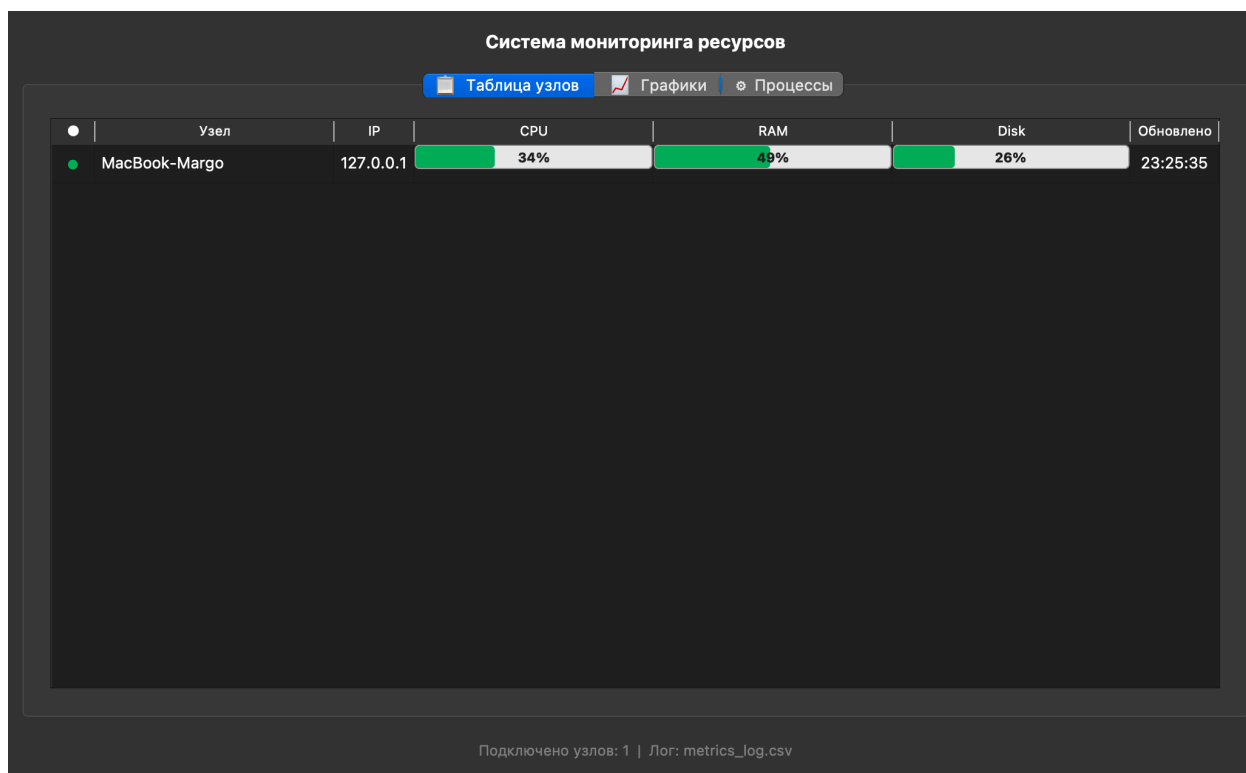


Рисунок 4.1 – Подключение одного агента (вкладка «Таблица узлов»)

На вкладке «Графики» автоматически сгенерировались три динамических графика. Результат успешного подключения представлен на рисунке 4.2.

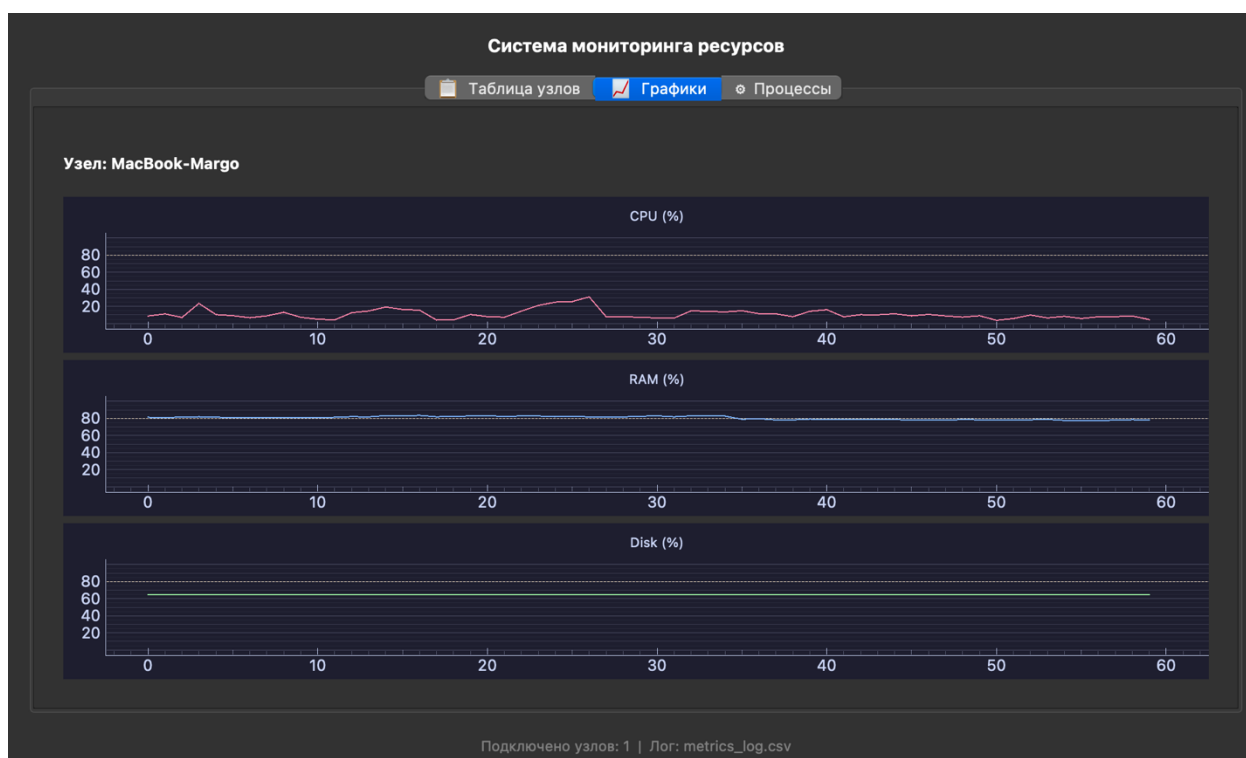


Рисунок 4.2 – Подключение одного агента (вкладка «Графики»)

Тест 2. Одновременное подключение нескольких агентов.

Были запущены два экземпляра агента с разными именами узлов. Сервер корректно принял оба подключения в отдельных потоках. В таблице отобразились две строки, на вкладке «Графики» – два независимых блока графиков. Тест пройден успешно. Результаты тестирования представлен на рисунках 4.3 и 4.4.

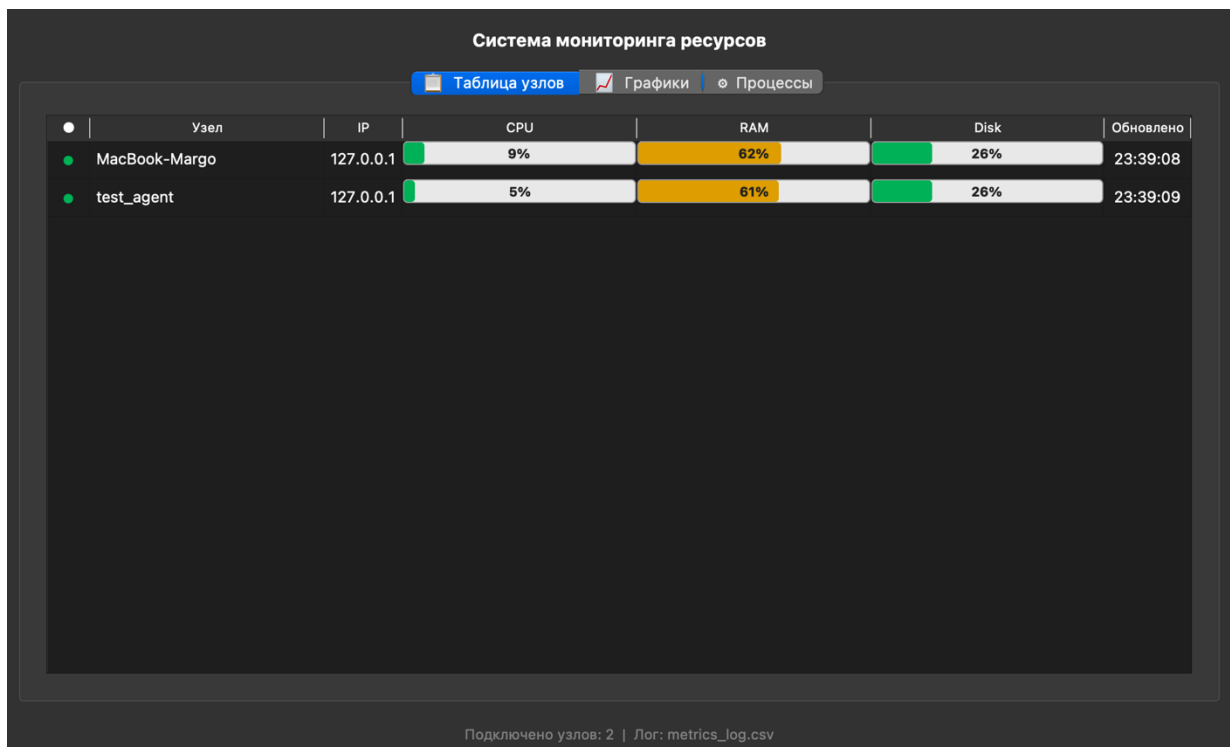


Рисунок 4.3 – Подключение нескольких агентов (вкладка «Таблица узлов»)

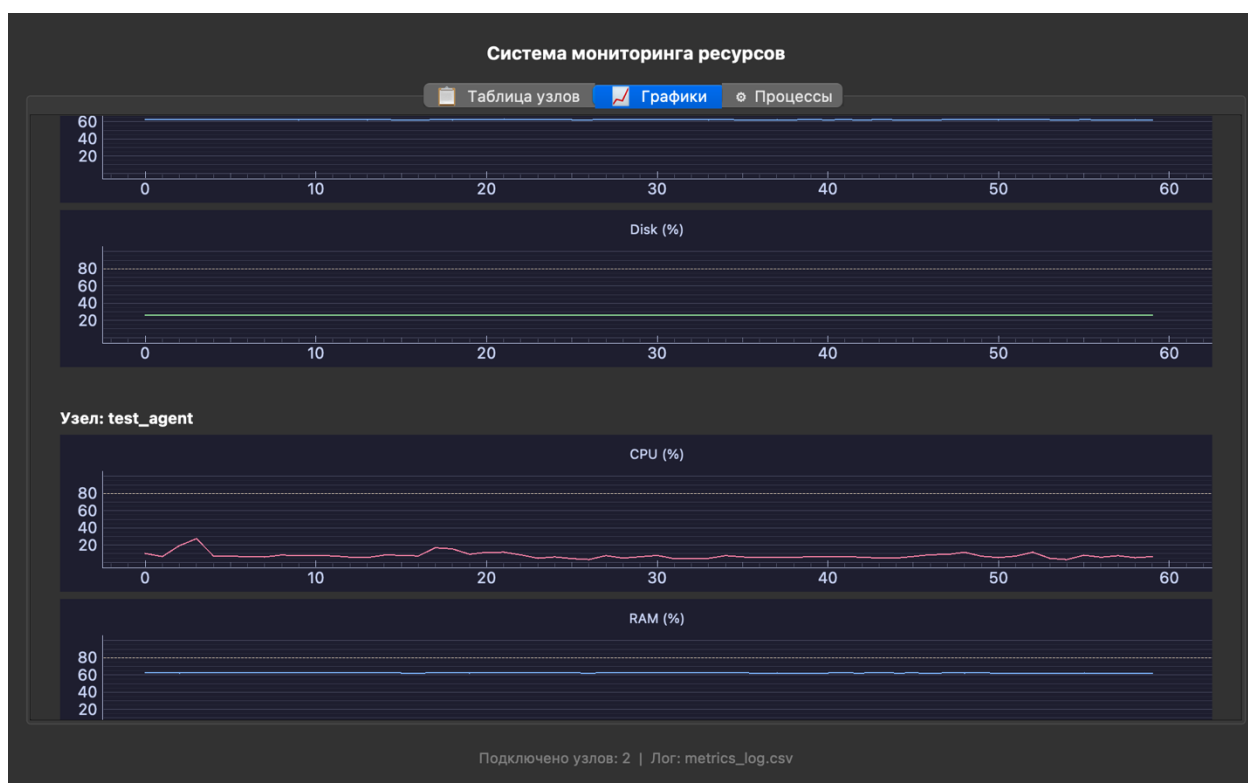


Рисунок 4.4 – Подключение нескольких агентов (вкладка «Графики»)

Тест 3. Проверка системы пороговых оповещений.

Для имитации высокой нагрузки значение порогового значения *CPU* было временно снижено до 5%. При следующем обновлении данных индикатор статуса изменил цвет с зелёного на красный, что свидетельствует о корректной работе системы оповещений. Результат тестирования представлен на рисунке 4.5.

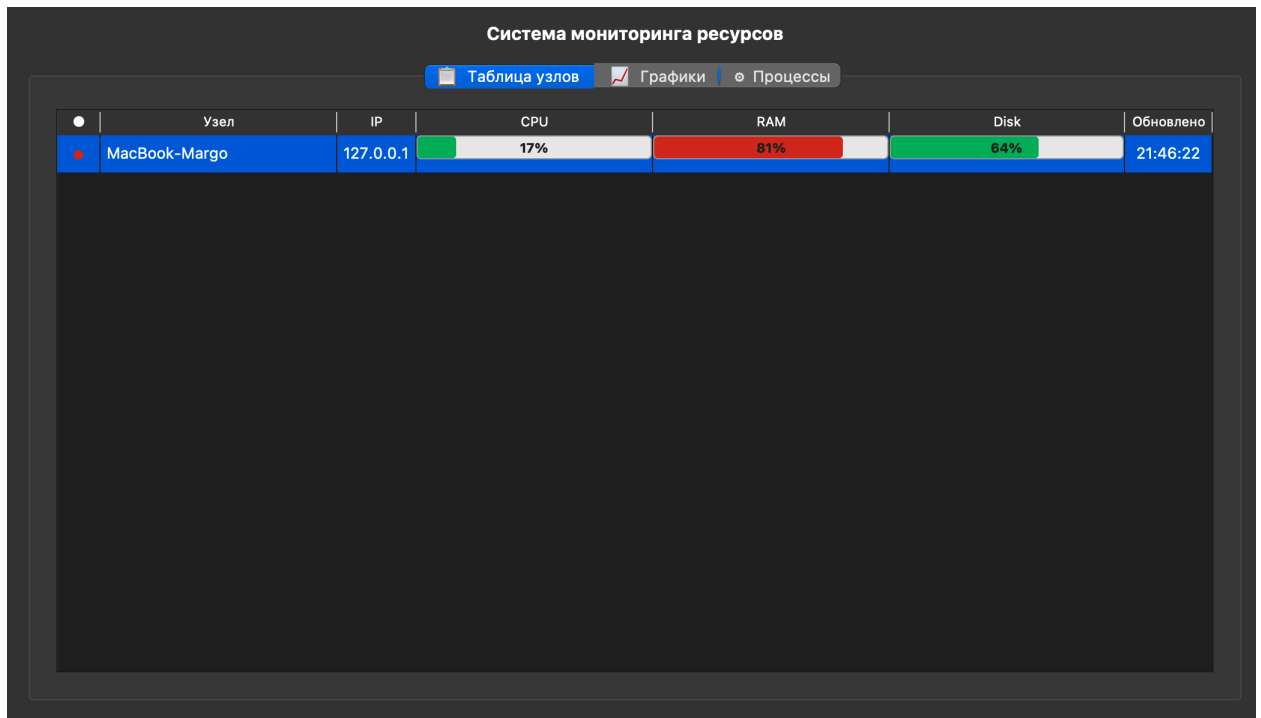


Рисунок 4.5 – Срабатывание оповещения при превышении порога

Тест 4. Проверка сохранения исторических данных (Логирование).

Для проверки механизма сохранения истории мониторинга система функционировала в штатном режиме в течение нескольких минут. После этого в рабочей директории сервера был открыт файл *metrics_log.csv*. В табличном файле корректно зафиксированы все полученные от агентов записи с указанием точного времени поступления пакета, имени узла, *IP*-адреса и конкретных значений аппаратных метрик (*CPU*, *RAM*, *Disk*). Данные структурированы корректно, разделители форматов соблюдены. Тест модуля логирования пройден успешно. Результаты на рисунке 4.6.

metrics_log

Время	Имя узла	IP адрес	CPU (%)	RAM (%)	Disk (%)
15:55:33	MacBook-Agent	127.0.0.1	18.8	70.4	46.5
15:55:36	MacBook-Agent	127.0.0.1	8.3	69.2	46.5
15:55:39	MacBook-Agent	127.0.0.1	11.3	69.1	46.5
15:55:42	MacBook-Agent	127.0.0.1	9.2	68.8	46.5
15:55:45	MacBook-Agent	127.0.0.1	11.9	70.4	46.5
15:55:48	MacBook-Agent	127.0.0.1	9.3	69.8	46.5
15:55:51	MacBook-Agent	127.0.0.1	19.1	70.4	46.5
15:55:54	MacBook-Agent	127.0.0.1	7.5	69.1	46.5
15:55:57	MacBook-Agent	127.0.0.1	18.0	69.9	46.5
15:56:00	MacBook-Agent	127.0.0.1	9.8	70.0	46.5
15:56:04	MacBook-Agent	127.0.0.1	7.9	70.0	46.5
15:56:07	MacBook-Agent	127.0.0.1	6.4	69.9	46.5
15:56:10	MacBook-Agent	127.0.0.1	8.7	69.6	46.5
15:56:13	MacBook-Agent	127.0.0.1	5.9	69.6	46.5
15:56:16	MacBook-Agent	127.0.0.1	8.4	69.6	46.5

Рисунок 4.6 – Файл истории метрик *metrics_log.csv*.

Тест 5. Обработка потери соединения. В ходе работы системы агент был принудительно завершён. Сервер корректно обработал разрыв соединения: вывел сообщение в консоль и перестал обновлять строку данного узла, не завершая при этом свою работу и сохраняя подключения других агентов. Тест пройден успешно. Разрыв соединения представлен на рисунке 4.7.

```
[+] Агент подключён: ('127.0.0.1', 49940)
[-] Агент ('127.0.0.1', 49940) отключился.
```

Рисунок 4.7 – Отображение разрыва соединения в консоли программы.

В целом все основные сценарии использования протестированы успешно. Разработанное программное средство корректно функционирует в условиях многопоточной работы и обеспечивает надёжный приём, отображение и сохранение метрик от нескольких узлов одновременно.

4.1 Анализ процесса установки соединения в *Wireshark*

Для подтверждения корректности работы сетевого взаимодействия на транспортном уровне использовался программный анализатор трафика *Wireshark*. Перехват пакетов осуществлялся на локальном интерфейсе (*Loopback*) с применением фильтрации по *TCP*-порту 9090. На рисунке 4.8 представлен процесс инициализации соединения между агентом и сервером.

Source	Destination	Protocol	Length	Info
127.0.0.1	127.0.0.1	TCP	68	57766 → 9090 [SYN] Seq=0 Win=65535 Len=0 MSS=16344 WS=64 TSval=2033686952 TSecr=0 SACK_PERM
127.0.0.1	127.0.0.1	TCP	68	9090 → 57766 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=16344 WS=64 TSval=3791598319 TSecr=203
127.0.0.1	127.0.0.1	TCP	56	57766 → 9090 [ACK] Seq=1 Ack=1 Win=408320 Len=0 TSval=2033686952 TSecr=3791598319
127.0.0.1	127.0.0.1	TCP	56	[TCP Window Update] 9090 → 57766 [ACK] Seq=1 Ack=1 Win=408320 Len=0 TSval=3791598319 TSecr=203
127.0.0.1	127.0.0.1	TCP	858	57766 → 9090 [PSH, ACK] Seq=1 Ack=1 Win=408320 Len=802 TSval=2033687986 TSecr=3791598319
127.0.0.1	127.0.0.1	TCP	56	9090 → 57766 [ACK] Seq=1 Ack=803 Win=407552 Len=0 TSval=3791599353 TSecr=2033687986
127.0.0.1	127.0.0.1	TCP	896	57766 → 9090 [PSH, ACK] Seq=803 Ack=1 Win=408320 Len=840 TSval=2033691020 TSecr=3791599353
127.0.0.1	127.0.0.1	TCP	56	9090 → 57766 [ACK] Seq=1 Ack=1643 Win=406720 Len=0 TSval=3791602387 TSecr=2033691020
127.0.0.1	127.0.0.1	TCP	904	57766 → 9090 [PSH, ACK] Seq=1643 Ack=1 Win=408320 Len=848 TSval=2033694051 TSecr=3791602387
127.0.0.1	127.0.0.1	TCP	56	9090 → 57766 [ACK] Seq=1 Ack=2491 Win=405888 Len=0 TSval=3791605418 TSecr=2033694051

Рисунок 4.8 – Установка соединения (Трёхкратное рукопожатие)

После успешного установления канала связи программный агент начинает циклическую отправку информации. Как видно из трафика, процесс «тройного рукопожатия» отрабатывает в строгом соответствии со спецификацией протокола *TCP*:

- в строке №1 агент отправляет серверу запрос на синхронизацию (пакет с флагом *[SYN]*), иницируя установку виртуального канала;
- в строке №2 сервер подтверждает получение запроса и отправляет встречную синхронизацию (пакет с флагами *[SYN, ACK]*);
- в строке №3 агент отправляет финальное подтверждение (пакет с флагом *[ACK]*), после чего соединение считается установленным и готовым к передаче данных.

4.2 Анализ отказоустойчивости системы (переподключение)

Важным аспектом сетевого мониторинга является отказоустойчивость системы при сбоях на линии или временной недоступности сервера, если произойдёт отключение сервера, то со стороны клиента будут происходить попытки переподключения до того момента пока не будет произведено успешное подключение. На рисунке 4.9 продемонстрирована работа механизма автоматического восстановления связи.

127.0.0.1	127.0.0.1	TCP	44	9090 → 57792 [RST] Seq=2 Win=0 Len=0
127.0.0.1	127.0.0.1	TCP	68	57797 → 9090 [SYN] Seq=0 Win=65535 Len=0 MSS=16344
127.0.0.1	127.0.0.1	TCP	44	9090 → 57797 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
127.0.0.1	127.0.0.1	TCP	68	57809 → 9090 [SYN] Seq=0 Win=65535 Len=0 MSS=16344
127.0.0.1	127.0.0.1	TCP	44	9090 → 57809 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
127.0.0.1	127.0.0.1	TCP	68	57810 → 9090 [SYN] Seq=0 Win=65535 Len=0 MSS=16344
127.0.0.1	127.0.0.1	TCP	44	9090 → 57810 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0

Рисунок 4.9 – Попытка переподключения агента к недоступному серверу

Анализ трафика в данной ситуации показывает следующее:

- агент пытается инициировать новое соединение, отправляя запрос на синхронизацию (пакет с флагом *[SYN]*);
- поскольку серверное приложение остановлено и порт 9090 закрыт, сетевой стек целевой операционной системы отклоняет запрос, мгновенно возвращая пакет с флагами *[RST, ACK]*;
- на уровне программного кода агента данное поведение сети вызывает исключительную ситуацию *ConnectionRefusedError*;
- агент корректно перехватывает ошибку, не завершает свою работу аварийно, а переходит в режим ожидания, циклически повторяя отправку *[SYN]*-запросов (каждые 3 секунды) до тех пор, пока сервер снова не станет доступен.

4.3 Анализ завершения сеанса связи

Корректное завершение работы приложений также фиксируется на уровне транспортного протокола. На рисунке 4.10 продемонстрирован процесс закрытия TCP-сессии при последовательном штатном отключении узлов системы (сначала завершается работа агента, затем – сервера).

127.0.0.1	127.0.0.1	TCP	56	9090 → 57811	[FIN, ACK] Seq=1 Ack=6692 Win=401792 Len=0 TSval=2638457019 TSecr=853338820
127.0.0.1	127.0.0.1	TCP	56	57811 → 9090	[ACK] Seq=6692 Ack=2 Win=408320 Len=0 TSval=853339217 TSecr=2638457019
127.0.0.1	127.0.0.1	TCP	902	57811 → 9090	[PSH, ACK] Seq=6692 Ack=2 Win=408320 Len=846 TSval=853341855 TSecr=2638457019
127.0.0.1	127.0.0.1	TCP	44	9090 → 57811	[RST] Seq=2 Win=0 Len=0

Рисунок 4.10 – Завершение соединения (*FIN, ACK*)

Анализ трафика демонстрирует классическую процедуру разрыва связи:

- при ручном завершении работы скрипта агента (*agent.py*), операционная система инициирует корректное закрытие клиентского сокета, отправляя серверу пакет с установленным флагом *[FIN, ACK]*;
- данный флаг сигнализирует о том, что передача данных со стороны клиента полностью завершена;
- серверный узел принимает этот пакет и отправляет встречное подтверждение *[ACK]*;
- последующее завершение работы серверного приложения (*server.py*) также сопровождается штатным освобождением сетевых портов и отправкой собственных пакетов завершения.

Такой механизм обмена управляющими пакетами гарантирует отсутствие «повисших» соединений и предотвращает утечки памяти в сетевом стеке операционной системы.

Таким образом, результаты тестирования в *Wireshark* доказывают, что разработанная архитектура работает надежно, устойчива к сбоям и полностью соответствует стандартам стека протоколов *TCP/IP*.

5 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

5.1 Системные и сетевые требования

Для обеспечения корректной работы сетевой системы мониторинга необходимо выполнение ряда минимальных системных и сетевых требований:

- операционная система – *Windows* 10/11 или *macOS* (программа кроссплатформенная);
- установлен *Python* версии 3.9 или новее;
- установлены библиотеки *PyQt5* (нужна серверу для графики и необязателен для скачивания, если нужен просто сбор данных с устройства) и *psutil* (нужна агенту для сбора данных). ;
- оба устройства должны находиться в одной локальной сети и поддерживать протоколы *TCP/IP*.

5.2 Настройка сетевого взаимодействия и параметров сокетов

Вся работа системы строится на передаче данных через *TCP*-сокеты. Чтобы агент и сервер смогли «увидеть» друг друга в сети, нужно правильно настроить параметры в файле *config.ini*. Этот файл создается сам при первом запуске агента, и его можно открыть любым текстовым редактором.

В файле нужно проверить следующие пункты:

1 Настройка сервера (секция *server*). Здесь параметр *ip* – это адрес компьютера, на котором запущен сервер. Если всё запускается на одном устройстве, оставляем 127.0.0.1. Параметр *port* (обычно 9090) – это номер порта, который сервер будет прослушивать через метод *listen*. Если *agent.py* будет запущен с другого устройства, то необходимо ввести *IP*-адрес сервера в *config.ini*.

2 Настройка брандмауэра. Нужно убедиться, что порт 9090 не заблокирован системой. Если порт закрыт, сокет выкинет ошибку *ConnectionRefusedError*, и данные не дойдут.

3 Параметры агента (секция *agent*). В поле *name* можно написать любое имя, чтобы понимать, чей это график. Параметр *send_interval* ставит задержку между отправками пакетов в секундах.

5.3 Порядок запуска

Для правильной установки связи (так называемого «рукопожатия» по *TCP*) важно соблюдать порядок включения программ:

1 Сначала запускаем *server.py*. Программа создаст сокет и привяжет его к адресу через метод *bind*. После этого сервер перейдет в режим ожидания клиентов, что и представлено на рисунке 5.1.



```
[*] Сервер запущен. Слушаю 0.0.0.0:9090...
```

Рисунок 5.1 – Запуск server.py

2 Затем запускаем *agent.py*. Агент создаст свой сокет и отправит запрос на подключение через метод *connect*. Если всё настроено верно, сервер примет соединение методом *accept* и начнет получать *JSON*-пакеты. Представление как будет выглядеть в консоли приложения представлено на рисунках 5.2 и 5.3.

```
[*] Агент запущен.  
[*] Узел: MacBook-Margo  
[*] Сервер: 127.0.0.1:9090  
[*] Интервал отправки: 2 сек.  
[*] Конфиг: /Users/margo/University 2nd Grad/Kursach/code/config.ini  
[+] Подключение установлено к 127.0.0.1:9090
```

Рисунок 5.2 – Сообщение об подключении на стороне agent.py

```
[+] Агент подключён: ('127.0.0.1', 56842)
```

Рисунок 5.3 – Сообщение об подключении на стороне server.py

5.4 Интерфейс программного средства

5.4.1 Главное окно

Сразу после того, как сервер запускается, на экране появляется основное окно. Оно устроено очень просто, чтобы в нем было легко ориентироваться. Сверху находится панель с тремя кнопками для переключения между разделами: «Таблица узлов», «Графики» и «Процессы» (рисунок 5.4).

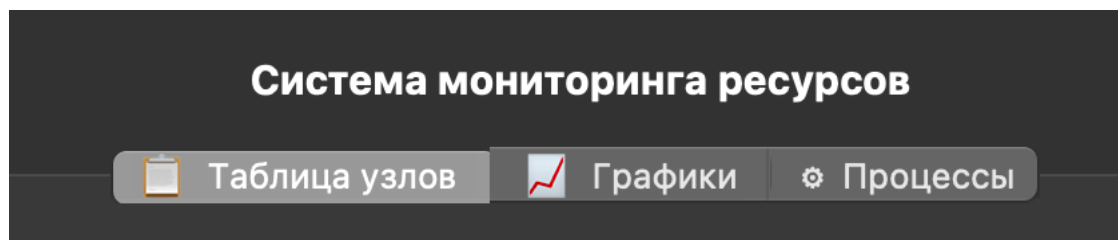


Рисунок 5.4 – Кнопки разделов

Нажимая на них, можно быстро переходить к нужным данным. В самом низу окна находится тонкая серая полоска – это строка статуса. Там программа пишет, сколько компьютеров сейчас подключено к серверу и как называется

файл, куда записывается история нагрузки (обычно это *metrics_log.csv*). На рисунке 5.5 это показано.

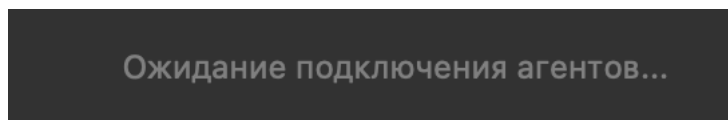


Рисунок 5.5 – Строка статуса

5.4.2 Вкладка «Таблица узлов»

Эту вкладку мы видим первой при запуске. Пока ни один агент не запущен, центральная часть окна будет просто пустой и темной (рисунок 5.6).

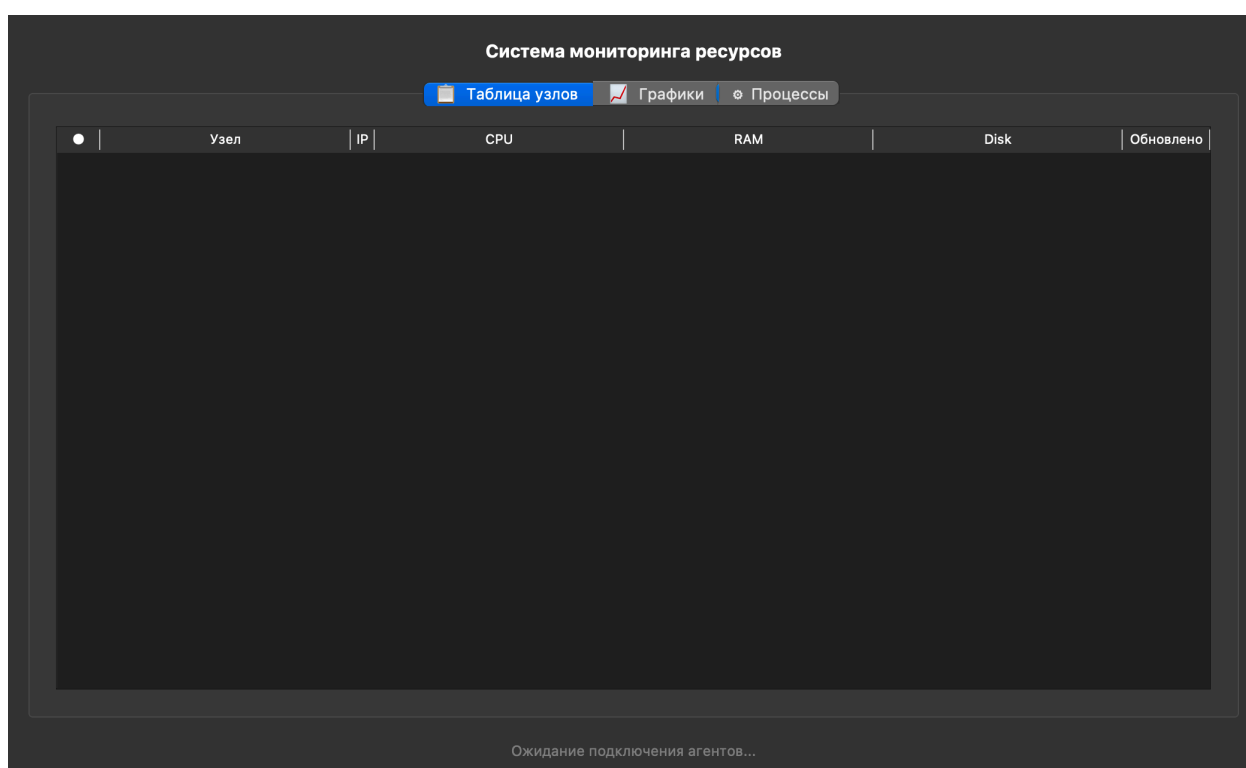


Рисунок 5.6 – Главное окно приложения

Но как только программа-агент на устройстве включается и связывается с сервером, здесь мгновенно появляется строка с данными.

Что именно мы видим в этой таблице:

- статус – маленький кружок в самом начале строки. Если всё работает нормально, он горит зеленым. Если компьютер начинает сильно тормозить или перегружаться, кружок становится красным;
- узел и IP – имя компьютера (например, *MacBook-Margo*) и его сетевой адрес, чтобы понимать, кто именно подключился;

– полоски *CPU*, *RAM*, *Disk* – цветные индикаторы нагрузки. Они показывают в процентах, насколько сейчас занят процессор или память. Данные на них обновляются сами каждые две секунды;

– обновлено – точное время, когда сервер последний раз получил информацию от этого устройства.

Если запустить несколько агентов на разных устройствах, в таблице просто появится еще одна строка ниже. Сервер будет следить за всеми сразу, не путая их данные между собой(рисунок 5.7).

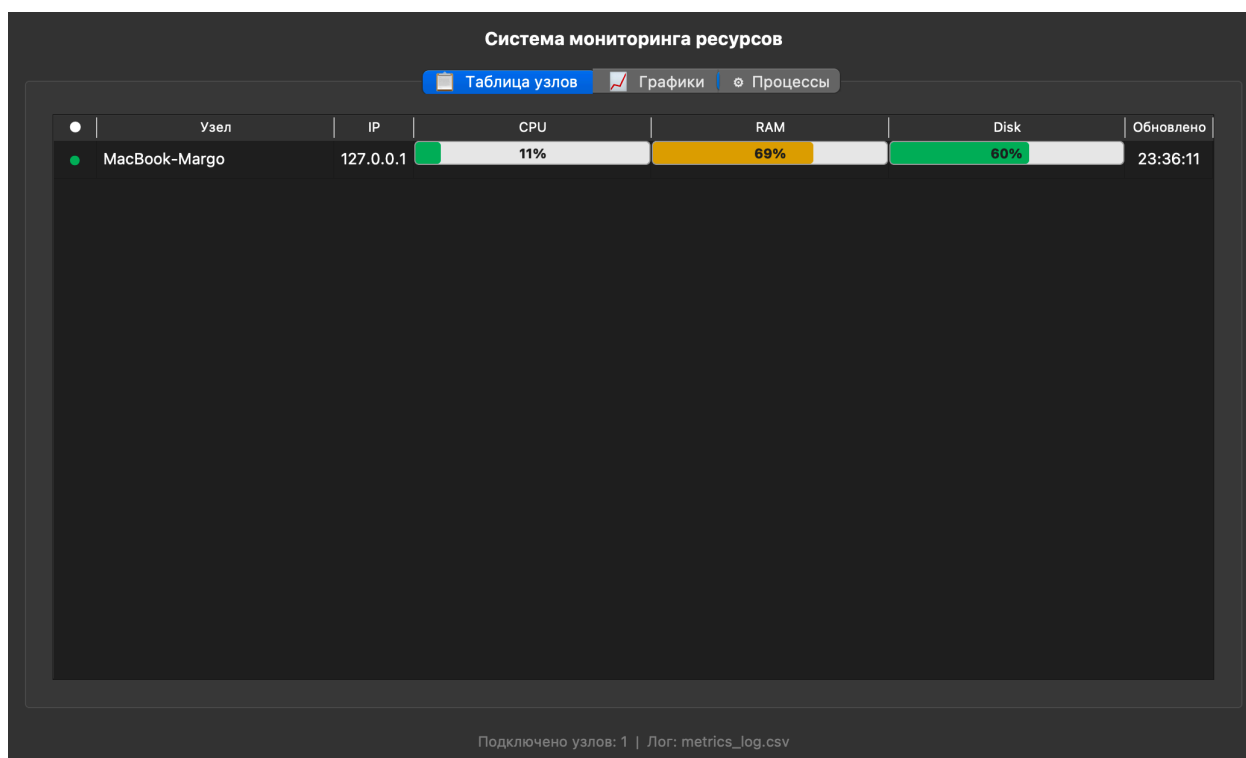


Рисунок 5.7 – Вкладка «Таблица узлов»

5.4.3 Вкладка «Графики»

Если нажать на кнопку «Графики», откроется раздел с живыми линиями нагрузки. Здесь для каждого подключенного устройства создается свой блок с тремя графиками. Розовая линия показывает «пульс» процессора, синяя – оперативную память, а зеленая – занятость диска.

Графики постоянно ползут влево, показывая историю за последнюю минуту. На каждом из них есть пунктирная красная линия – это граница, выше которой компьютер считается перегруженным. Если агентов несколько, их блоки графиков будут располагаться друг под другом, и их можно будет просто прокручивать вниз(рисунок 5.8).

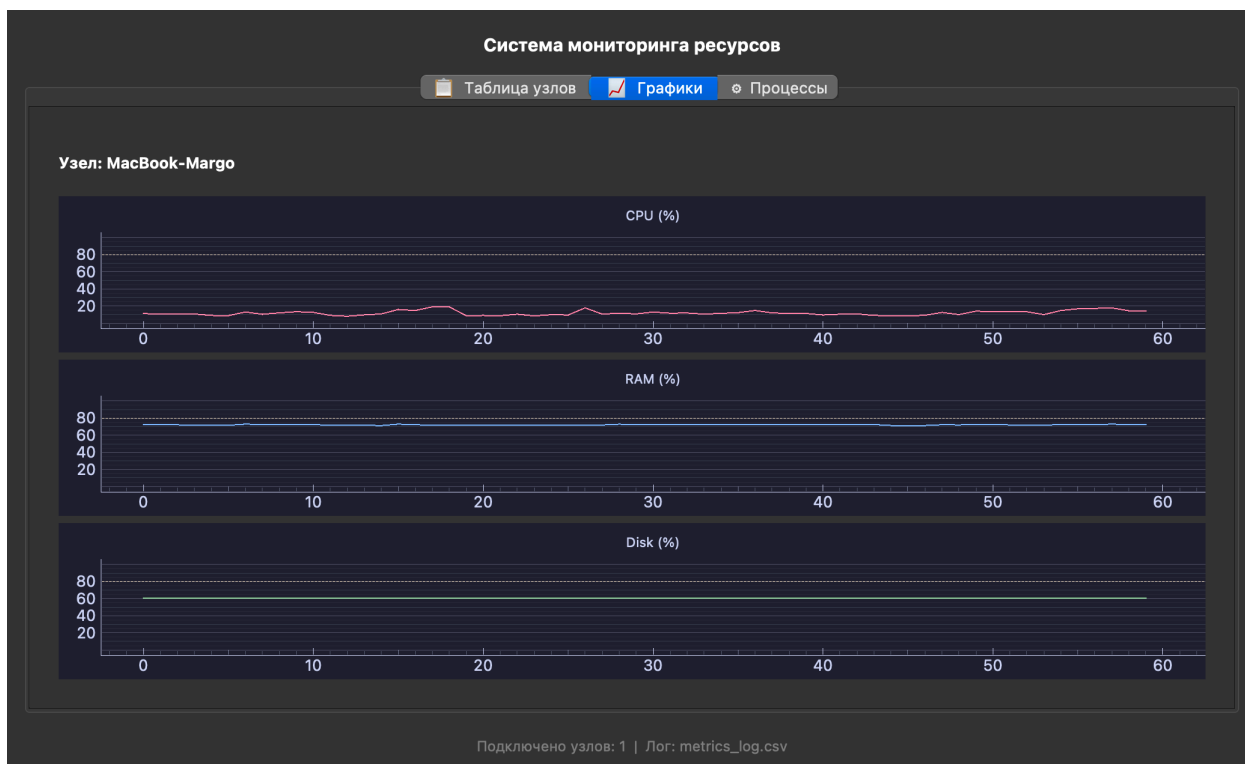


Рисунок 5.8 – Вкладка «Графики»

5.4.4 Вкладка «Процессы»

Этот раздел помогает найти конкретную программу, которая «съедает» все ресурсы устройства. Чтобы увидеть данные, нужно сначала кликнуть на выпадающий список сверху и выбрать имя нужного компьютера.

После выбора устройства на экране появятся две таблицы:

1 левая таблица показывает топ-5 программ, которые сильнее всего нагружают процессор в данный момент;

2 правая таблица показывает приложения, которые занимают больше всего оперативной памяти.

Это очень удобно, если в таблице узлов загорелся красный индикатор – можно сразу перейти сюда и увидеть, из-за какой программы это произошло (рисунок 5.9).

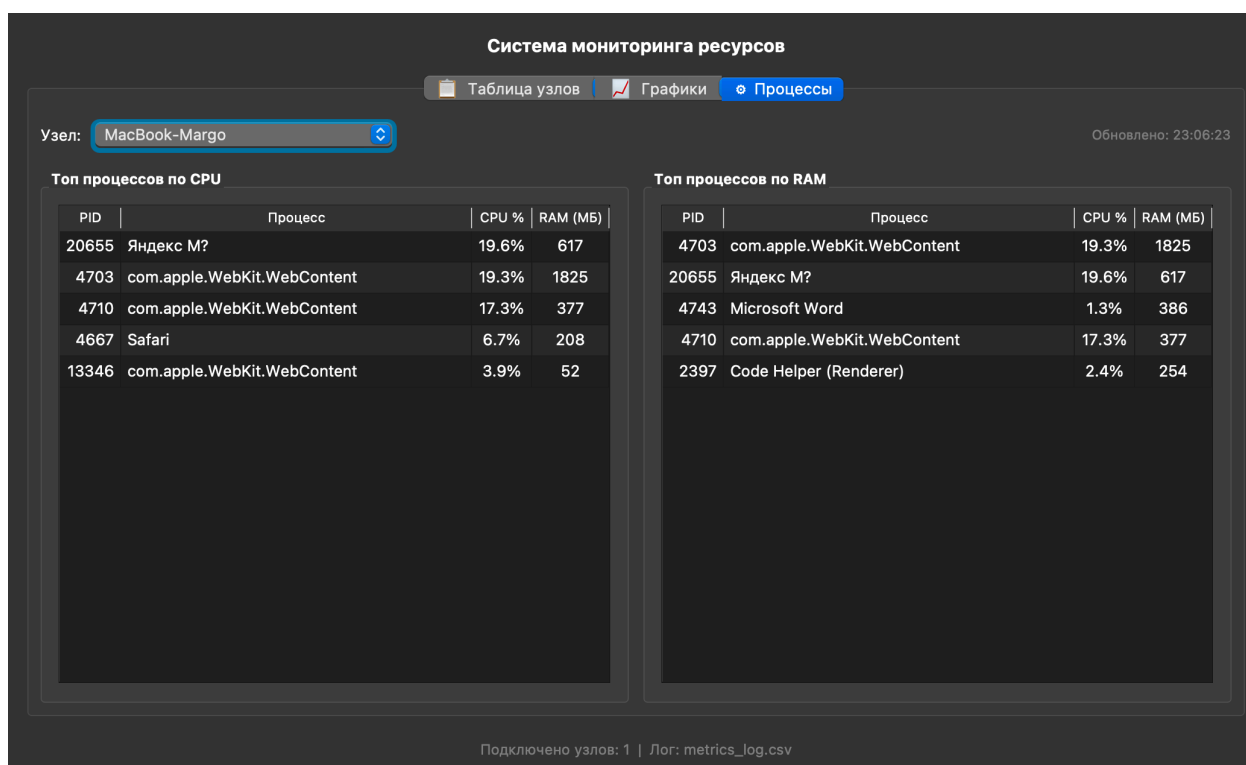


Рисунок 5.9 – Вкладка «Процессы»

Чтобы завершить работу сервера, достаточно нажать на кнопку закрытия приложения. И сервер прекратит свою работу.

Благодаря грамотному разделению функционала на три независимые вкладки, администратор получает исчерпывающий набор инструментов для контроля сети. «Таблица узлов» обеспечивает общий мониторинг в режиме реального времени, «Графики» позволяют оценивать динамику нагрузки, а вкладка «Процессы» дает возможность точно выявлять проблемные приложения. Таким образом, разработанная система является удобным, наглядным и эффективным инструментом, полностью решающим поставленные задачи сетевого мониторинга.

ЗАКЛЮЧЕНИЕ

Данная курсовая работа была посвящена созданию собственной системы мониторинга с нуля – задаче, которая охватывает сразу несколько ключевых областей: сетевое программирование, многопоточность, проектирование *GUI* и хранение данных. Разработанное программное средство обеспечивает сбор, передачу, отображение и сохранение системных метрик в режиме реального времени.

В ходе работы была глубоко изучена тема сетевого взаимодействия на основе *TCP*-сокетов. Работа с низкоуровневым *API* модуля *socket* дала понимание того, как устроена передача данных в сети, которую в обычных условиях скрывают высокоуровневые фреймворки.

Сравнение с промышленными аналогами – *Zabbix* и *Prometheus* – показало, что для небольших сетей громоздкая инфраструктура избыточна. Разработанное решение при минимальных зависимостях (*PyQt5*, *psutil*, *pyqtgraph*) решает ту же задачу: собирает метрики нескольких узлов, строит живые графики, ведёт *CSV*-журнал и сигнализирует об аномалиях. Кроссплатформенность подтверждена запуском на *macOS*.

- сводная таблица с цветовыми индикаторами и прогресс-барами;
- живые графики метрик на основе библиотеки *pyqtgraph*;
- вкладка истории с данными из базы *SQLite*.

Таким образом, в рамках курсового проекта были выполнены все поставленные задачи: реализован агент с автоматическим переподключением и сбором топ-процессов, многопоточный сервер с *GUI*, *CSV*-логирование и система пороговых оповещений. Все компоненты протестированы в пяти тестах.

Практическая ценность разработанной курсовой работы заключается в ее универсальности, кроссплатформенности и предельной легкости развертывания. Для запуска системы мониторинга не требуются глубокие знания баз данных или конфигурации веб-серверов – достаточно наличия интерпретатора *Python*, что делает данное решение идеальным для использования в малых локальных сетях.

Данный продукт будет полезен для тех, кому необходимо производить мониторинг данных со своего устройства или, например в кооперативном случае – со всех подключённых устройств, в режиме реального времени.

Перспективы развития: добавление шифрования канала (*TLS*), мониторинг сетевых интерфейсов через *psutil.net_io_counters()*, веб-интерфейс на *Flask* для доступа без установки клиента, интеграция с *Telegram Bot API* для мгновенных оповещений.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Стандарт предприятия. Требования к оформлению пояснительных записок к курсовым проектам. – Минск : БГУИР, 2025. – 42 с.
- [2] Официальная документация Python 3: модуль socket [Электронный ресурс]. – Режим доступа : <https://docs.python.org/3/library/socket.html>. – Дата доступа : 12.05.2026.
- [3] Официальная документация Python 3: модуль json [Электронный ресурс]. – Режим доступа : <https://docs.python.org/3/library/json.html>. – Дата доступа : 12.05.2026.
- [4] Официальная документация Python 3: модуль threading [Электронный ресурс]. – Режим доступа : <https://docs.python.org/3/library/threading.html>. – Дата доступа : 12.05.2026.
- [5] Официальная документация PyQt5 [Электронный ресурс]. – Режим доступа : <https://www.riverbankcomputing.com/static/Docs/PyQt5/>. – Дата доступа : 12.05.2026.
- [6] Официальная документация psutil [Электронный ресурс]. – Режим доступа : <https://psutil.readthedocs.io/en/latest/>. – Дата доступа : 12.05.2026.
- [7] Официальная документация pyqtgraph [Электронный ресурс]. – Режим доступа : <https://pyqtgraph.readthedocs.io/en/latest/>. – Дата доступа : 12.05.2026.
- [8] Zabbix Documentation 7.0 [Электронный ресурс]. – Режим доступа : <https://www.zabbix.com/documentation/7.0/en/manual>. – Дата доступа : 10.05.2026.
- [9] Prometheus Documentation: Overview [Электронный ресурс]. – Режим доступа : <https://prometheus.io/docs/introduction/overview/>. – Дата доступа : 10.05.2026.
- [10] Таненбаум Э. Компьютерные сети / Э. Таненбаум, Д. Уэзеролл. – 5-е изд. – СПб. : Питер, 2012. – 960 с.
- [11] Stevens W. R. UNIX Network Programming. Volume 1: The Sockets Networking API / W. R. Stevens, B. Fenner, A. M. Rudoff. – 3rd ed. – Addison-Wesley, 2003. – 1024 p.
- [12] RFC 793 – Transmission Control Protocol [Электронный ресурс] / J. Postel. – IETF, 1981. – Режим доступа : <https://www.rfc-editor.org/rfc/rfc793>. – Дата доступа : 12.05.2026.
- [13] RFC 8259 – The JavaScript Object Notation (JSON) Data Interchange Format [Электронный ресурс] / T. Bray. – IETF, 2017. – Режим доступа : <https://www.rfc-editor.org/rfc/rfc8259>. – Дата доступа : 12.05.2026.
- [14] IEEE 802.3-2022 – IEEE Standard for Ethernet [Электронный ресурс]. – Режим доступа : <https://standards.ieee.org/ieee/802.3/10422/>. – Дата доступа : 12.05.2026.

ПРИЛОЖЕНИЕ А

Исходный код программы

agent.py:

```
import socket
import json
import time
import psutil
import platform
import configparser
import os

CONFIG_FILE = os.path.join(os.path.dirname(__file__),
                             'config.ini')

def load_config():
    config = configparser.ConfigParser()

    if not os.path.isfile(CONFIG_FILE):
        # Создаём config.ini автоматически при первом запуске
        config['server'] = {
            'ip': '127.0.0.1',
            'port': '9090',
        }
        config['agent'] = {
            'name': platform.node(),
            'send_interval': '2',
            'top_n': '5',
        }
        with open(CONFIG_FILE, 'w', encoding='utf-8') as f:
            config.write(f)
        print(f"[*] Создан файл конфигурации: {CONFIG_FILE}")
        print(f"[*] Отредактируй его при необходимости и перезапусти агент.")
    else:
        config.read(CONFIG_FILE, encoding='utf-8')

    return config

config = load_config()

SERVER_IP = config.get('server', 'ip',
                       fallback='127.0.0.1')
SERVER_PORT = config.getint('server', 'port',
                             fallback=9090)
NODE_NAME = config.get('agent', 'name',
                       fallback=platform.node())
SEND_INTERVAL = config.getint('agent', 'send_interval',
                               fallback=2)
TOP_N = config.getint('agent', 'top_n',
                      fallback=5)
```

```

DISK_PATH = '/' if platform.system() != 'Windows' else 'C:\\'

# -----
# Сбор метрик процессов
# -----

def get_top_processes(n=TOP_N):
    """
    Возвращает два списка:
    top_cpu - топ-N процессов по загрузке CPU
    top_ram - топ-N процессов по использованию RAM (в МБ)

    Почему try/except внутри цикла:
    - процесс может завершиться прямо во время итерации ->
    NoSuchProcess
    - системные процессы не дают читать свои данные ->
    AccessDenied
    - memory_info может вернуть None на macOS ->
    проверяем явно
    """
    procs = []

    for p in psutil.process_iter(['pid', 'name',
    'cpu_percent', 'memory_info']):
        try:
            mem_info = p.info['memory_info']

            # Явная проверка на None - на macOS некоторые
            системные
            # процессы возвращают memory_info=None вместо
            объекта

            if mem_info is None:
                continue

            mem_mb = round(mem_info.rss / 1024 / 1024, 1)

            procs.append({
                'pid': p.info['pid'],
                'name': p.info['name'] or 'unknown',
                'cpu': round(p.info['cpu_percent'] or 0.0,
1),
                'ram_mb': mem_mb,
            })

        except (psutil.NoSuchProcess, psutil.AccessDenied,
psutil.ZombieProcess):
            pass

    top_cpu = sorted(procs, key=lambda x: x['cpu'],
reverse=True)[:n]
    top_ram = sorted(procs, key=lambda x: x['ram_mb'],
reverse=True)[:n]

```

```

        return top_cpu, top_ram

# -----
# Сбор системных метрик
# -----

def collect_metrics():
    top_cpu, top_ram = get_top_processes()

    data = {
        'node': NODE_NAME,
        'cpu': psutil.cpu_percent(interval=1),
        'ram': psutil.virtual_memory().percent,
        'disk': psutil.disk_usage(DISK_PATH).percent,
        'top_cpu': top_cpu,
        'top_ram': top_ram,
    }
    return data

# -----
# Основной цикл с автопереподключением
# -----

def start_agent():
    print(f"[*] Агент запущен.")
    print(f"[*] Узел: {NODE_NAME}")
    print(f"[*] Сервер: {SERVER_IP}:{SERVER_PORT}")
    print(f"[*] Интервал отправки: {SEND_INTERVAL} сек.")
    print(f"[*] Конфиг: {CONFIG_FILE}")

    while True:
        # — Фаза 1: подключение (повторяем до успеха) —
        s = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
        connected = False

        while not connected:
            try:
                s.connect((SERVER_IP, SERVER_PORT))
                print(f"[+] Подключение установлено к
{SERVER_IP}:{SERVER_PORT}")
                connected = True
            except ConnectionRefusedError:
                print(f"[!] Сервер {SERVER_IP}:{SERVER_PORT}
недоступен. "
                    f"Повтор через 3 сек...")
                time.sleep(3)
            except OSError:
                s.close()
                s = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
                time.sleep(3)

```



```

# — Фаза 2: цикл отправки метрик —
try:
    while True:
        data = collect_metrics()
        json_data = json.dumps(data,
ensure_ascii=False) + '\n'

        s.sendall(json_data.encode('utf-8',
errors='replace'))
        top_name_raw = data['top_cpu'][0]['name'] if
data['top_cpu'] else 'n/a'
        top_name = top_name_raw.encode('utf-8',
errors='replace').decode('utf-8')

        print(f">]                                CPU={data['cpu']}%
RAM={data['ram']}% "
                f"Disk={data['disk']}% | топ CPU:
{top_name}")

        time.sleep(SEND_INTERVAL)

    except (BrokenPipeError, ConnectionResetError,
OSError) as e:
        print(f"[-] Связь потеряна: {e}.
Переподключение...")
        finally:
            s.close()

        time.sleep(2)

if __name__ == '__main__':
    start_agent()

```

server.py:

```

import socket
import json
import time
import psutil
import platform
import configparser
import os

# _____
# Чтение конфига
# _____
# Файл config.ini должен лежать рядом с agent.py
# Если файла нет - создаём его с настройками по умолчанию

CONFIG_FILE = os.path.join(os.path.dirname(__file__),
'config.ini')

def load_config():

```

```

config = configparser.ConfigParser()

if not os.path.isfile(CONFIG_FILE):
    # Создаём config.ini автоматически при первом запуске
    config['server'] = {
        'ip': '127.0.0.1',
        'port': '9090',
    }
    config['agent'] = {
        'name': platform.node(),
        'send_interval': '2',
        'top_n': '5',
    }
    with open(CONFIG_FILE, 'w', encoding='utf-8') as f:
        config.write(f)
    print(f"[*] Создан файл конфигурации: {CONFIG_FILE}")
    print(f"[*] Отредактируй его при необходимости и
перезапусти агент.")
else:
    config.read(CONFIG_FILE, encoding='utf-8')

return config

config = load_config()

SERVER_IP = config.get('server', 'ip',
fallback='127.0.0.1')
SERVER_PORT = config.getint('server', 'port',
fallback=9090)
NODE_NAME = config.get('agent', 'name',
fallback=platform.node())
SEND_INTERVAL = config.getint('agent', 'send_interval',
fallback=2)
TOP_N = config.getint('agent', 'top_n',
fallback=5)

DISK_PATH = '/' if platform.system() != 'Windows' else 'C:\\'
# -----
# Сбор метрик процессов
# -----

def get_top_processes(n=TOP_N):
    """
    Возвращает два списка:
    top_cpu - топ-N процессов по загрузке CPU
    top_ram - топ-N процессов по использованию RAM (в МБ)

    Почему try/except внутри цикла:
    - процесс может завершиться прямо во время итерации ->
NoSuchProcess
    - системные процессы не дают читать свои данные ->
AccessDenied

```

```

        - memory_info может вернуть None на macOS
проверяем явно
        """
        procs = []

        for p in psutil.process_iter(['pid', 'name',
            'cpu_percent', 'memory_info']):
            try:
                mem_info = p.info['memory_info']

                # Явная проверка на None - на macOS некоторые
системные
                # процессы возвращают memory_info=None вместо
объекта

                if mem_info is None:
                    continue

                mem_mb = round(mem_info.rss / 1024 / 1024, 1)

                procs.append({
                    'pid': p.info['pid'],
                    'name': p.info['name'] or 'unknown',
                    'cpu': round(p.info['cpu_percent'] or 0.0,
1),
                    'ram_mb': mem_mb,
                })

            except (psutil.NoSuchProcess, psutil.AccessDenied,
psutil.ZombieProcess):
                pass

        top_cpu = sorted(procs, key=lambda x: x['cpu'],
reverse=True)[:n]
        top_ram = sorted(procs, key=lambda x: x['ram_mb'],
reverse=True)[:n]

        return top_cpu, top_ram

# -----
# Сбор системных метрик
# -----

def collect_metrics():
    top_cpu, top_ram = get_top_processes()

    data = {
        'node': NODE_NAME,
        'cpu': psutil.cpu_percent(interval=1),
        'ram': psutil.virtual_memory().percent,
        'disk': psutil.disk_usage(DISK_PATH).percent,
        'top_cpu': top_cpu,
        'top_ram': top_ram,
    }
    return data

```

```

# -----
# Основной цикл с автопереподключением
# -----

def start_agent():
    print(f"[*] Агент запущен.")
    print(f"[*] Узел: {NODE_NAME}")
    print(f"[*] Сервер: {SERVER_IP}:{SERVER_PORT}")
    print(f"[*] Интервал отправки: {SEND_INTERVAL} сек.")
    print(f"[*] Конфиг: {CONFIG_FILE}")

    while True:
        # — Фаза 1: подключение (повторяем до успеха) —
        s = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
        connected = False

        while not connected:
            try:
                s.connect((SERVER_IP, SERVER_PORT))
                print(f"[+] Подключение установлено к
{SERVER_IP}:{SERVER_PORT}")
                connected = True
            except ConnectionRefusedError:
                print(f"[!] Сервер {SERVER_IP}:{SERVER_PORT}
недоступен. "
                    f"Повтор через 3 сек...")
                time.sleep(3)
            except OSError:
                s.close()
                s = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
                time.sleep(3)

        # — Фаза 2: цикл отправки метрик —
        try:
            while True:
                data = collect_metrics()
                json_data = json.dumps(data,
ensure_ascii=False) + '\n'

                s.sendall(json_data.encode('utf-8',
errors='replace'))

                top_name_raw = data['top_cpu'][0]['name'] if
data['top_cpu'] else 'n/a'
                top_name = top_name_raw.encode('utf-8',
errors='replace').decode('utf-8')

                print(f"[>] CPU={data['cpu']}%
RAM={data['ram']}% "

```

```

        f"Disk={data['disk']}%      | топ CPU:
{top_name}")

        time.sleep(SEND_INTERVAL)

    except (BrokenPipeError, ConnectionResetError,
            OSError) as e:
        print(f"[-] Связь потеряна: {e}.
Переподключение...")
        finally:
            s.close()

        time.sleep(2)

if __name__ == '__main__':
    start_agent()

```

config.ini:

```

[server]
ip = 127.0.0.1
port = 9090

[agent]
; Имя этого узла - будет отображаться в интерфейсе сервера
; По умолчанию берётся имя компьютера из системы
name = MacBook-Margo

; Как часто отправлять данные на сервер (в секундах)
send_interval = 2

; Сколько процессов показывать в топе
top_n = 5

```

ПРИЛОЖЕНИЕ Б
Блок-схема алгоритма

